

ENGG2112 Multi-Disciplinary Engineering

Semester 2, 2026
Week 4

Professor T. J. Lim



THE UNIVERSITY OF
SYDNEY

Celebrating 175 years

CRICOS 00026A TEQSA PRV12057



What We're Covering Today

Data Pre-Processing or Cleaning

Stochastic Gradient Descent

Classical Machine Learning Methods – K nearest neighbours, random forest, naïve Bayes

- Training
- Parameters versus Hyper-Parameters
- Python implementation

Data Pre-Processing

Let's keep it clean!

Importing Data

- Recall that we have a table of data...
- Some items might be missing...
- What do we do?

Features or labels (K)

Samples or records
(N)

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Timestamp	Gearbox_T1_High_Speed_Shaft_Temperature	Gearbox_T3_High_Speed_Shaft_Temperature	Gearbox_T1_Intermediate_Speed_Shaft_Temperature	Temperature Gearbox Bearing Hollow Shaft	Tower Acceleration Normal	Gearbox_Oil-2_Temperature	Tower Acceleration Lateral	Temperature Bearing_A	Temperature Trafo-3	Gearbox_1	Gearbox_2	Gearbox_3
2	1/01/2019 0:00	57	59	52	56.15833282	47.05377579	57	18.8907719	35	48.57666779	56.15833	55	55
3	1/01/2019 0:10	57.69333267	59.48500061	52.45000076	57	64.96974182	57.41333389	28.5602646	35.2333336	48	57	55.175	55.175
4	1/01/2019 0:20	59	60.75666809	53.53666687	57.77500153	51.14966965	58.72833252	34.2288132	36	48.05333328	57.775	56	56
5	1/01/2019 0:30	59.88166809	61.5633316	54.41333389	58.68333435	58.74092865	59.51833344	35.5932198	36	48.78833389	58.68333	56.73833	56.73833
6	1/01/2019 0:40	61.29000092	62.58666611	55.48500061	59.62333298	53.26477432	60.66500092	38.5527306	36	49	59.62333	57.615	57.615
7	1/01/2019 0:50	62.27166748	63.76833344	56.64500046	60.78333282	60.29292297	61.79833221	37.0367241	36.9016685	49.25999832	60.78333	58.70667	58.70667
8	1/01/2019 1:00	63	64.3666687	57.25666809	61.61333466	55.85303116	62.45333481	34.3804169	37	50	61.61333	59.555	59.555
9	1/01/2019 1:10	62.875	64.09333038	57.78333282	62	51.71955109	62.0616684	30.2212811	36.2666664	50	62	60	60
10	1/01/2019 1:20	63	64.94499969	58	62	48.2154541	63	37.6760826	35.0050011	50	62	60	60
11	1/01/2019 1:30	63.65833282	65.31666565	58.63499832	62.76499939	51.44218063	63.56499863	41.2617722	35.0116653	50	62.76333	60.67333	60.67333
12	1/01/2019 1:40	64.00666809	66	59	62.44400060	47.34379841	62.07000121	36.7865240	36	50	62.445	61.00	61.00

Dealing with Missing Data

- When analysing real data, we very often encounter missing data, i.e. some cells are blank because the data was not recorded properly. E.g.

55	1/01/2019 8:50	64.63834	67	59.57833	64	49.32137	65.20333	47.2467	36.66333	72.71333	64
56	1/01/2019 9:00	64.74	67	59.37667	64	40.65483	65	44.42561	36	72	64
57	1/01/2019 9:10	65.245	67.33	60	64	42.17761	65.52	38.45386	36	72	64
58	1/01/2019 9:20	65.64833	68	60	64.83334	58.79045	66	34.65066	36	72	64.83167
59	1/01/2019 9:30	65.06667	67.415	60	64.51167	61.63845	65.62666	44.45857	36	72	64.51167
60	1/01/2019 9:40	65	67.11	60	64	53.69125	65.34666	34.76271	36		64
61	1/01/2019 9:50	65	67.41	60	64	53.35581	65.82	45.37477	36		64
62	1/01/2019 10:00	65.385	67.565	60.15167	64.245	67.75699	65.675	46.91055	36		64.245
63	1/01/2019 10:10	66	68	61	65	52.71021	66	41.84181	35.61833		65
64	1/01/2019 10:20	66	68	61	65	59.43783	66	44.32015	35		65
65	1/01/2019 10:30	65.96833	68	61	65	49.32779	66	45.15066	35		65
66	1/01/2019 10:40	65	67.61667	60.03	65	51.08233	65.68333	43.39077	34.92833		65
67	1/01/2019 10:50	64.63	67	60	64.11833	56.68871	65	41.90113	34.01667		64.11833
68	1/01/2019 11:00	63.445	66.355	58.98667	63.74833	54.67158	64.01334	51.97982	34		63.74833
69	1/01/2019 11:10	61.39333	64.54	57.39833	62.38334	50.2406	62.31	51.70904	33.175		62.38334

What to do with these cells?

Pandas tools for data pre-processing

- Any missing data is converted to NaN (Not a Number) when loading a file using pandas.

```
import pandas as pd
X = pd.read_csv("../Data/Power-missing-data.csv")
X.head(10)
```

9]:

	Timestamp	Power(kW)
0	1/01/2019 0:00	705.876648
1	1/01/2019 0:10	884.711670
2	1/01/2019 0:20	NaN
3	1/01/2019 0:30	1115.943359
4	1/01/2019 0:40	1263.841675
5	1/01/2019 0:50	1249.811646
6	NaN	1136.536621
7	1/01/2019 1:10	966.803345
8	1/01/2019 1:20	1091.823364
9	1/01/2019 1:30	1234.765015

- NaN is frequently treated as zero, e.g. summing, averaging, etc.
- This may or may not be sensible of course.
- If we want to ignore rows that have any missing data, we use the dropna function:

```
import pandas as pd
X = pd.read_csv("../Data/Power-missing-data.csv")
X.dropna().head()
```

2]:

	Timestamp	Power(kW)
0	1/01/2019 0:00	705.876648
1	1/01/2019 0:10	884.711670
3	1/01/2019 0:30	1115.943359
4	1/01/2019 0:40	1263.841675
5	1/01/2019 0:50	1249.811646

Pandas tools for data pre-processing

- To find the missing entries in a data frame, we use the isna function. The opposite of this is notna.

```
import pandas as pd
X = pd.read_csv("../Data/Power-mi")
X.isna().head()
```

5]:

	Timestamp	Power(kW)
0	False	False
1	False	False
2	False	True
3	False	False
4	False	False

```
import pandas as pd
X = pd.read_csv("../Data/Power")
X.notna().head()
```

5]:

	Timestamp	Power(kW)
0	True	True
1	True	True
2	True	False
3	True	True
4	True	True

Pandas tools for data pre-processing

- We can even fill in the NaN values using a variety of methods through the fillna function.
- Ref: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html#pandas.DataFrame.fillna>

```
import pandas as pd
X = pd.read_csv("../Data/Power-missing-d")
X.fillna(method="ffill").head()
```

]:

	Timestamp	Power(kW)
0	1/01/2019 0:00	705.876648
1	1/01/2019 0:10	884.711670
2	1/01/2019 0:20	884.711670
3	1/01/2019 0:30	1115.943359
4	1/01/2019 0:40	1263.841675

Forward
Fill

```
import pandas as pd
X = pd.read_csv("../Data/Power-missing-dat")
X.fillna(value={"Power(kW)":200}).head()
```

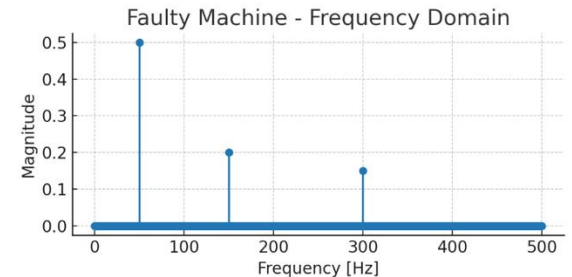
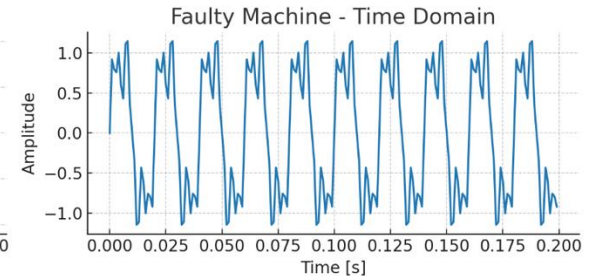
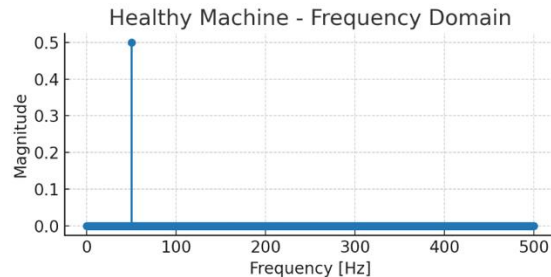
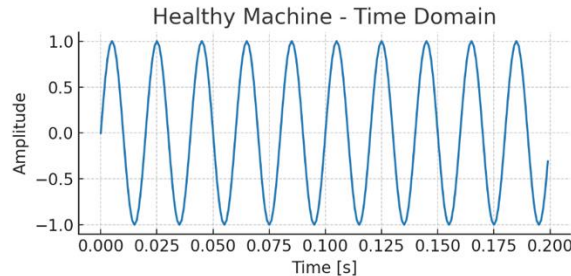
]:

	Timestamp	Power(kW)
0	1/01/2019 0:00	705.876648
1	1/01/2019 0:10	884.711670
2	1/01/2019 0:20	200.000000
3	1/01/2019 0:30	1115.943359
4	1/01/2019 0:40	1263.841675

NaN values in the
Power(kW)
column filled with
200

Other forms of data pre-processing

- Engineering data often comes in the form of signals measured at regular intervals of time.
- Important information is often in the frequency domain, with the main differences between two classes appearing as frequency artefacts.
- Common to “transform” : time-domain $\leftrightarrow$ frequency domain, using the Fourier transform (or some other transform).



Other forms of data pre-processing

- Break a long sequence of measurements into short, appropriately sized “windows”.

Extracting dominant features from signals to differentiate classes is more an art than a science. Using the entire signal can be inefficient and we can often reduce dimensionality through feature engineering.

- **Feature engineering** is often the step that takes the most time in an ML project.
 - Feature Selection (picking the best inputs) – looking for the highest correlation with the output
 - Variance threshold (remove nearly-constant features)
 - In training separation using Decision Tree or Random Forest ML methods first
 - Using PCA (Principal Component Analysis) -> captures maximum variance
 - Manual feature engineering using human insights into data

Imbalanced Datasets

Sometimes, one class is much more prevalent than others, skewing ML training. **This is called imbalanced data.**

The trained model will be optimised for finding the majority case

E.g. if 95% of the data is “blue” and the performance metric is accuracy, then the model may output “blue” independent of the input and maximize performance.

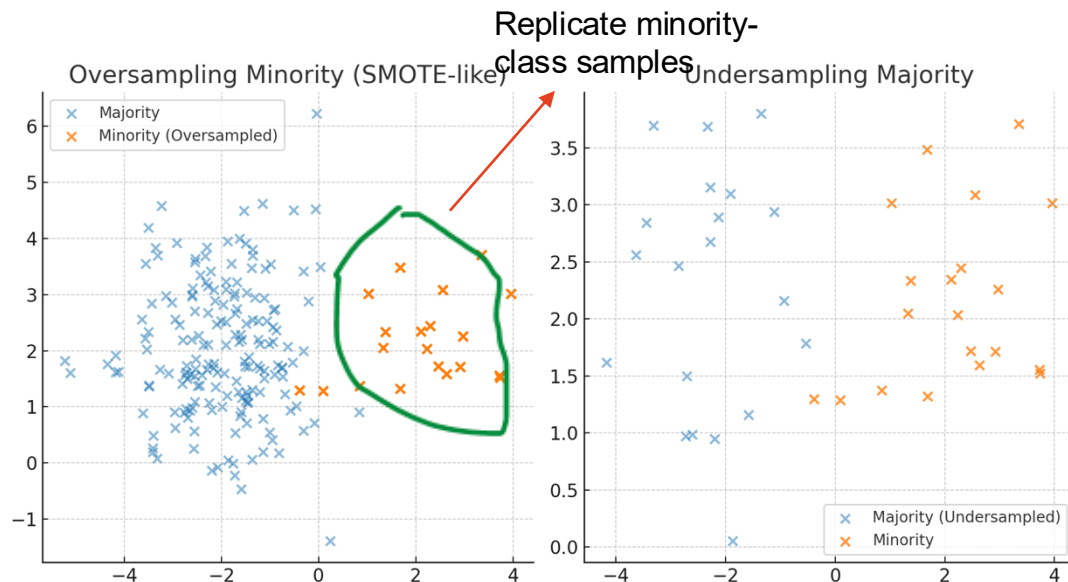
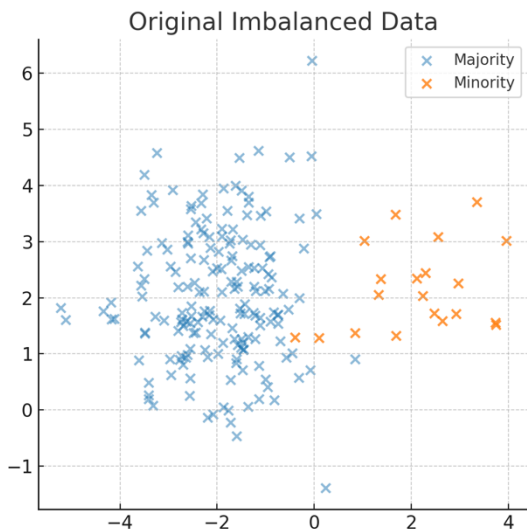
Therefore, we need to prevent that.



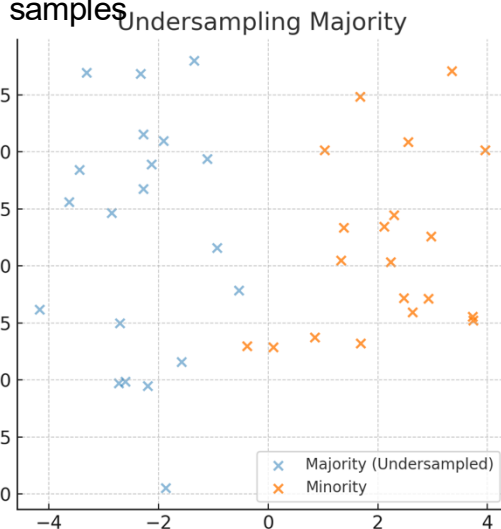
Resampling of Imbalanced Data

Resampling methods:

- Oversampling minority class (e.g., SMOTE – **S**ynthetic **M**inority **O**versampling **T**echnique).
- Undersampling the majority class.



Replicate minority-class samples

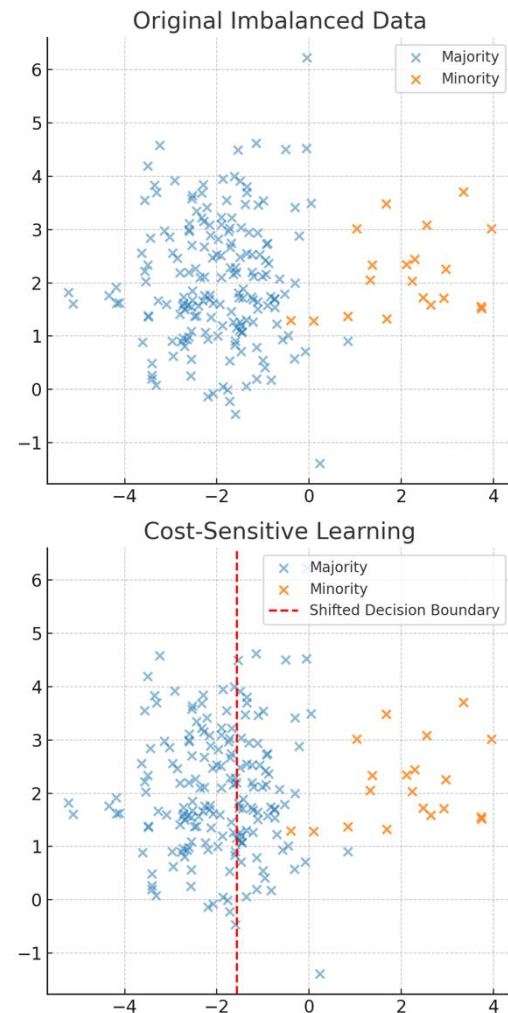


Other Ways to Handle Imbalanced Data

Cost-Sensitive Learning

- Assign higher penalties to costly errors during training.
- Define a new loss function that reflects the relative costs of FP and FN.
- ML model adapts by **shifting decision boundary** to reduce the more expensive error.
- Goal: **Minimise overall cost**, not just improve accuracy.

Data augmentation: Create synthetic but realistic minority class data.



One-Hot Encoding of Categorical Variables

- Features may not be numerical, e.g. airport names, airlines over on the right.
- Most ML algorithms expect numerical inputs though (otherwise you can't multiply, etc.)
- The most basic method for converting such categorical variables into a numeric representation is "one-hot encoding".
- We first look at how many values the categorical variable can take, say M .
- Then we create $M-1$ new "dummy" 0/1 variables, with a 1 in the position corresponding to the category.

	D	E	F	
	AirportFro	AirportTo	DayOfWee	Ti
269	SFO	IAH	3	
1558	PHX	CLT	3	
2400	LAX	DFW	3	
2466	SFO	DFW	3	
108	ANC	SEA	3	
1094	LAX	IAH	3	
1768	LAX	MSP	3	
2722	PHX	DTW	3	
2606	SFO	MSP	3	
2538	LAS	ORD	3	
223	ANC	SEA	3	
1646	PHX	ATL	3	

One-Hot Encoding of Categorical Variables

- Example: Suppose Gender is one of the features, and the allowable values are Male/Female/Not Given. The problem is to estimate the amount spent in a given shop based on gender (as one of the features).
- Then we generate two dummy variables, x_1 and x_2 to represent Male and Female respectively. Note that there's no need for a 3rd dummy variable because $x_1 = x_2 = 0$ means Gender is in the 3rd category.
- In the following table, customers 1 and 2 are male, 3 and 4 are female, and 5 is unknown.

Customer	Male (x_1)	Female (x_2)	Expenditure (\$ y)
1	1	0	100
2	1	0	120
3	0	1	95
4	0	1	150
5	0	0	200

One-Hot Encoding of Categorical Variables

- With this, we can now perform regression or classification using the new dummy variables.
- **Question:** Could we have simply assigned say 0 to Male, 1 to Female and 2 to Unknown? What impact would that have had?
 - Answer: No, that would be risky and imply an ordinal relationship between the genders, i.e. Male < Female < Unknown.

One-Hot Encoding in Python

- This is the sklearn implementation using the function OneHotEncoder
- Returns results in a numpy array, which can be converted into a pandas dataframe if necessary.

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd

V = pd.read_csv("../Code/LightDutyVehicles.csv")
# Initialize encoder with the right properties, e.g. dropping one variable so that
# there are only k-1 variables for an input with k classes by specifying the optional
# argument drop="first".
enc = OneHotEncoder()
# enc2 = OneHotEncoder(drop="first")

# Encode the "Category" and "Manufacturer" columns of the V dataframe and save it as a numpy
# array.
V_encoded = enc.fit_transform(V[["Category", "Manufacturer"]]).toarray()
Cat = enc.categories_ # A list of two arrays, each containing the category names from the two
# encoded columns.

Vdf = pd.DataFrame(V_encoded[:, :6], columns = Cat[0])
Vdf2 = pd.DataFrame(V_encoded[:, 6:], columns = Cat[1])
V_out = pd.concat([Vdf, Vdf2], axis=1)
V_out
```

✓ 0.1s

Python

	Passenger Van/Shuttle Bus	Pickup	SUV	Sedan/Wagon	Van	Vocational/Cab Chassis	Acura	Audi	BMW	BYD Motors	...
0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...
1	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...
2	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...

One-Hot Encoding in Python

- This implementation uses pandas to create a dataframe with one-hot encoded columns.

```
import pandas as pd
v2 = pd.get_dummies(V["Category"], prefix="Cat")
v2.head(10)
#v3 = v2.to_numpy()
#print(v3)
```

|:

	Cat_Passenger Van/Shuttle Bus	Cat_Pickup	Cat_SUV	Cat_Sedan/Wagon	Cat_Van	Cat_Vocational/Cab Chassis
0	0	0	0	1	0	0
1	0	0	0	1	0	0
2	0	0	0	1	0	0
3	0	1	0	0	0	0
4	0	1	0	0	0	0
5	0	0	0	0	1	0
6	0	0	0	0	1	0
7	0	0	0	1	0	0
8	0	0	0	1	0	0
9	0	0	0	1	0	0

Classical Machine Learning

Everything but neural networks

Mathematical Foundations of Classification and Regression

- Assume we don't know the relationship between x and y (or else the model is already known!).
- For a given input x , there is some chance that the output y takes a certain value.
 - E.g. Consider that x encodes the words “The quick brown fox” and y is the next word in the sentence. We might have probabilities
 - $P[y = \text{“jumps”} \mid x] = 0.4$; $P[y = \text{“walks”} \mid x] = 0.1$; $P[y = \text{“is”} \mid x] = 0.05$, etc.
 - If we knew these conditional probabilities, then which output y would we choose if forced to choose only one?
- Clearly, a detector that knows $P[y \mid x]$ can output the y that maximizes this function.
 - In detection theory, this is known as the *maximum a posteriori* (MAP) detector.
 - For discrete y (classification), MAP minimizes the probability of making a decision error, i.e. it **maximizes accuracy**.
- However, how do we find the conditional distribution $P[y \mid x]$?

Mathematical Foundations of Classification and Regression

- Either it needs to be learned from training data, or it is derived mathematically from knowledge or assumptions about the system being modelled.
 - Consider the transmission of a bit over a noisy channel. The bit value is either 0 or 1, and it is received as either 0 or 1, but there is a probability of bit error p , i.e.
 - $P[0 \text{ received} \mid 1 \text{ transmitted}] = P[1 \text{ received} \mid 0 \text{ transmitted}] = p$
 - If p is known through measurements, then we have the required conditional distribution.
- In other cases, there isn't such a simple model that would adequately match reality.
 - In many applications, especially those involving the mimicking of human cognitive ability, we don't have sufficient understanding of the underlying process to design a simple accurate model.
 - That's why we have general AI/ML models that (sort of) works in many scenarios.
- We will examine some of the simpler ones next.

Did Probabilistic Foundations Influence AI Model Design?

- The short answer is YES.
- Explicitly applied in the Naïve Bayes model, which is basically computing the MAP decision with several major simplifying assumptions.
- Indirectly manifested in the "soft" outputs created by the softmax function (more on this later).
- The long answer is more nuanced.
 - AI/ML has also been developed from the computer science algorithms angle, as we will see when we discuss random forest and K-nearest neighbours (KNN).
 - However, they still have been designed to output probabilities.

Naïve Bayes Model

- Assume K features $x_1, \dots, x_K$ and one discrete feature y . This is a classification problem.
- Bayes rule applied here says

$$p(y|x_1, \dots, x_K) = \frac{p(x_1, \dots, x_K|y)p(y)}{p(x_1, \dots, x_K)}$$

- For a given vector of observed features $x_1, \dots, x_K$, we want to calculate the distribution of the label y using the RHS, then pick the value of y that maximizes this distribution.
- The denominator is irrelevant (just a normalizing constant).
- $p(y)$ can be found from training data.
- But $p(x_1, \dots, x_K|y)$ is hard to figure out. This is the joint distribution of the features given the label. Given training data, we would in theory have to estimate the relative frequency of each possible $p(x_1, \dots, x_K|y)$:
 - If the features are binary, there will be 2^K such relative frequencies – there is usually not enough data to get good estimates of the required probabilities.

Gaussian Naïve Bayes Model – Independence and Gaussian Assumptions

- The key to progressing to a general inference model are two simplifying assumptions: (a) that all the features are conditionally independent and (b) all the individual conditional feature distributions are Gaussian.
- Assumption (a) allows us to write

$$p(x_1, \dots, x_K | y) = \prod_{i=1}^K p(x_i | y)$$


Gaussian

- While Assumption (b) means finding the mean and variance of each feature conditioned on y gives us its distribution.

$$p(x_i | y) = \frac{1}{\sigma_{i,y}} \exp\left(-\frac{(x_i - \mu_{i,y})^2}{2\sigma_{i,y}^2}\right)$$

Gaussian NB

- For each class $y \in \mathcal{S}_y$, find mean and variance of each corresponding feature. E.g. if $y \in \{0,1\}$, we may have:

y	x_1	x_2
0	0.2	0.9
0	-0.6	1.2
1	2.4	-1.0
1	3.1	-0.6
1	2.8	-1.4



$$\mu_{1,0} = -0.2$$

$$\mu_{2,0} = 1.05$$

$$\mu_{1,1} = 2.77$$

$$\mu_{2,1} = -1.0$$

Conditional variances can be estimated from the data too.

Thus, we have the Gaussian distributions needed.

Gaussian NB in Python

```
From sklearn.naive_bayes import GaussianNB

nb = GaussianNB() # Give the model a name
nb.fit(X_train, y_train) # Train the model on training data
y_pred = nb.predict(X_test) # Generate predictions from test
                             data
```

That's it really! In scikit-learn, almost all ML models are trained using `<xyz>.fit` and inference using the model invokes `<xyz>.predict`, where `<xyz>` is the name of your model.

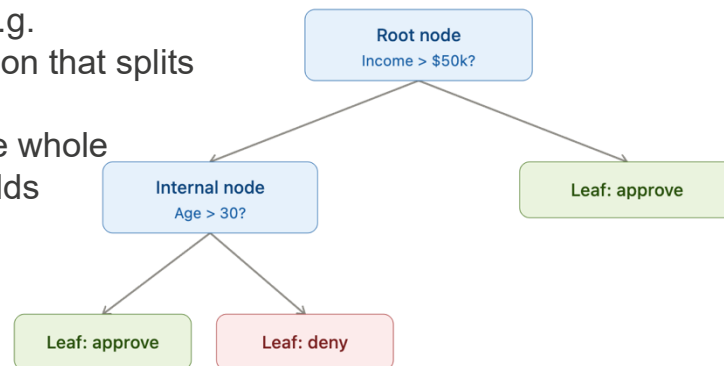
Upon first definition, `<xyz>` needs to be given the right hyper-parameter values (in GNB, there happen to be none).

Gaussian NB Parameters

- From the training data, we estimate the conditional means and variances of every feature conditioned on each class label.
- These are the **parameters** of the Gaussian NB model.
- There are no hyper-parameters as the structure of the Gaussian NB model is entirely determined by the number of features and the number of classes the label can take.

Random Forest – Decision Tree Concept

- Unlike the Gaussian NB, the random forest is a model derived from a computer science perspective.
- Based on concept of a decision tree, e.g.
- At each level of a tree, we ask a question that splits the data into two parts
- For a given data sample, traversing the whole tree from top (root) to bottom (leaf) yields the final decision at the leaf node.
- In a random forest, we run in parallel a number of trees, then take a majority decision.



Random Forest – Specifications

- For each decision tree, the following are the key hyper-parameters set by us humans:
 - a. The maximum depth (levels) of the tree (`max_depth`);
 - b. How many features to choose from at each node (`max_features`);
 - c. The quality measure used in splitting (`criterion`).
- In the training phase the following parameters are optimized by the machine:
 - a. Which feature to split on at each node;
 - b. Threshold value to use for the split at each node.
- The main idea is that at each node, ideal split of data results in only one class label per branch – in that case, no further splitting is needed. We just need to find:
 - Which feature to use at each node, and what threshold to use, that would bring us as “close” to that ideal as possible.
 - The measure of closeness is the `criterion` hyper-parameter – either a Gini impurity or entropy.
- We will illustrate this in class with a worked example.

Random Forest – Randomization is Key

- Problem with relying on a single tree is that it may overfit the training data, i.e. perform very well on the training data but not generalize.
- In a random forest, we avoid this problem by using many decision trees, each one randomized (and so different from each other) in the following ways:
 - **Bagging (bootstrap sampling)**, i.e. each tree trains on a different resampled dataset.
 - **Random feature subsets** at each split, i.e. every node is allowed to only investigate m out of p features, chosen randomly and independently.
- The randomization prevents different trees turning out the same as each other and defeating the purpose of introducing multiple trees.
 - Rather clever!

Random Forests and Python 😊

```
from sklearn.ensemble import RandomForestClassifier

# Create Random Forest model with chosen hyper-parameters
RF = RandomForestClassifier(n_estimators = 100, max_depth = 5)

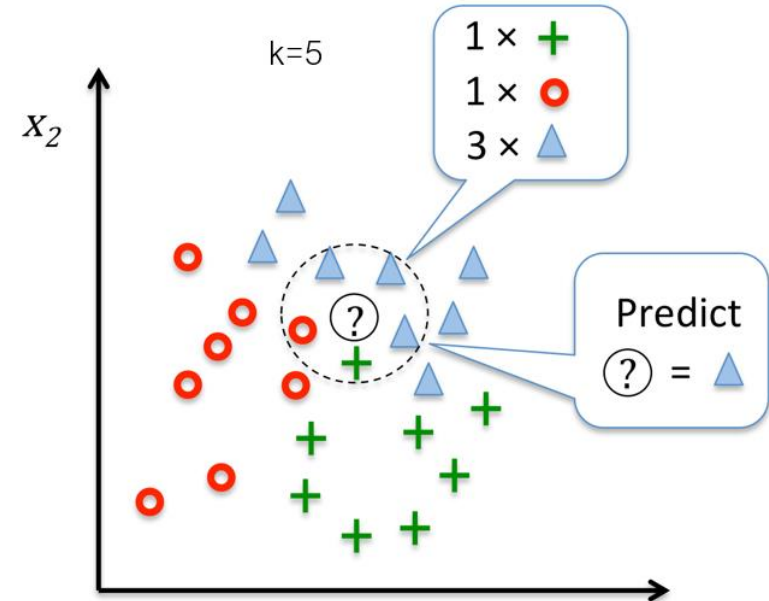
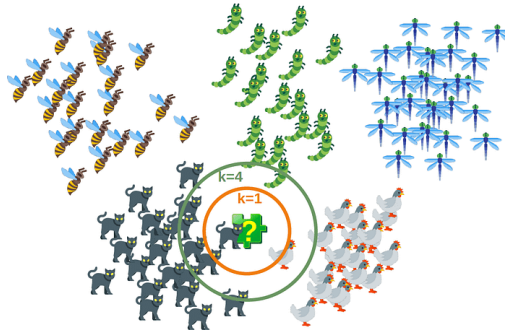
# Train with training data
RF.fit(X_train, y_train)

# Infer labels in test data
Y_pred = RF.predict(X_test)

# Compute performance, plot charts, etc.
...
```

Supervised Machine learning: K Nearest Neighbour

- We have **N training samples** (x_i, y_i) $i = 1 \dots N$.
- y_i are **class labels**: they take on a limited number of possible values.
- *In the example, each point belongs to a category such as “bee,” “cat,” or “dragonfly.”*
- X_i are **feature vectors**: in this case, each has **two dimensions**, so we can represent them on a 2D Cartesian plane.



<https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4>

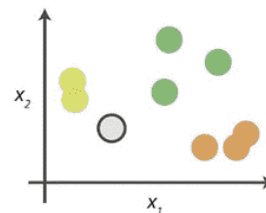
Supervised Machine learning: K Nearest Neighbour

1. Set parameter K to an odd value, say 5.
2. Denote training feature vectors by $\mathbf{x}_i^{(m)}$, where $m \in \mathcal{M}$ is the class label (or category) and i the sample index.
3. For a new feature vector $\mathbf{x}$, compute the N distances where N is the number of training data records:

$$d_i(\mathbf{x}) = \left| \mathbf{x} - \mathbf{x}_i^{(m)} \right|^2, \quad i = 1, \dots, N$$

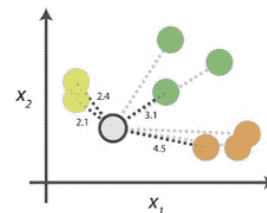
4. Sort from smallest to largest, and note the K smallest values and their class labels.
5. Classifier output is the class label with highest count.

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point Distance	
...	2.1 → 1st NN
...	2.4 → 2nd NN
...	3.1 → 3rd NN
...	4.5 → 4th NN

Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

3. Vote on labels

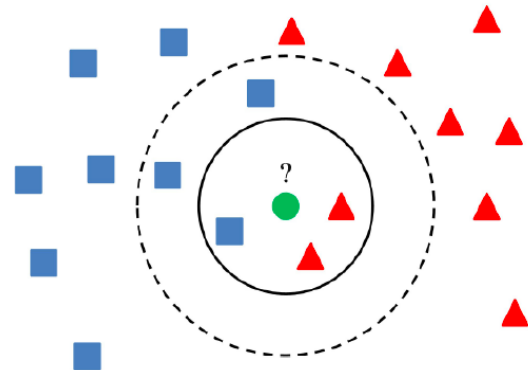
Class	# of votes
	2
	1
	1

Class wins the vote!
Point is therefore predicted to be of class .

Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the $k=3$ nearest neighbours.

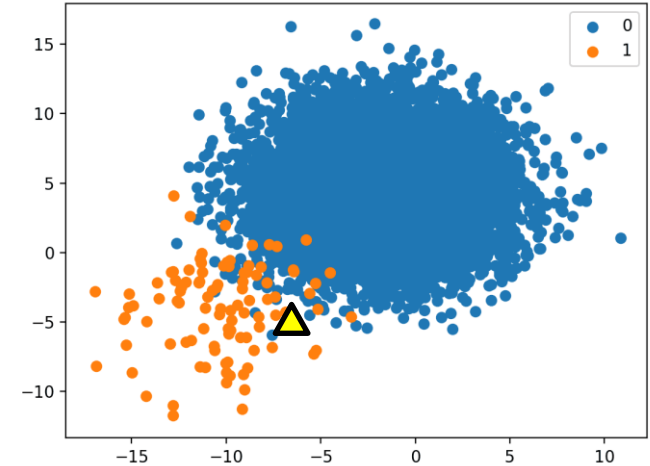
Supervised Machine learning: K Nearest Neighbour

- In an illustrated example, when $K=3$, the classifier output is “triangle”.
- But when $K = 5$, classifier output is “square”.
- Choice of K is important but there is no formulaic way to choose it well for any given data set.
 - Note that K does not impact **computational complexity** – we always have to compute all N distances. It does impact **classifier accuracy**.
 - As complexity grows with N , best not to use “too many” training samples.
- Distance measure need not be L_2 (Euclidean). May also be Hamming (= no. of positions where two vectors differ), L_1 , etc.



KNN: Imbalanced Data

- We can have an imbalanced data problem, i.e. one class dominates the training set, e.g. 90% of the training data are of class 0, only 10% of class 1. (See *example on right*)
- Suppose test feature is the **yellow triangle**. Though visually it looks like it's in the orange class, KNN may return an output of blue because there are more blue neighbours!
- Solution: Give nearer neighbours higher “weight” in the count.

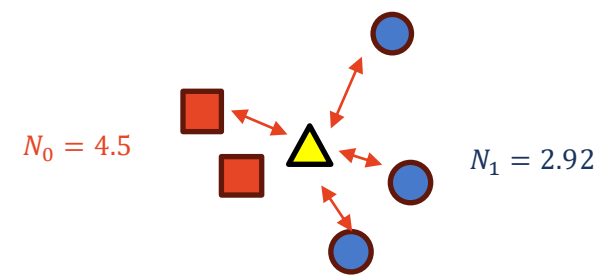


Weighted Nearest Neighbour

To be precise:

1. Find the K training points closest to the test point and their distances to the test point, $d_i(\mathbf{x})$, $i \in \mathcal{I}_K(\mathbf{x})$, where $\mathcal{I}_K(\mathbf{x})$ is the set of K training indices having smallest distance to $\mathbf{x}$.
2. Group those K points according to class. For class n , compute $N_n = \sum_i \frac{1}{d_i(\mathbf{x})}$, where we're only summing over the points in class n .
3. Classifier output is the value of n that maximizes N_n .

Weighted Nearest Neighbour



- Example: Suppose $K = 5$, and we have the following distances from the test point to the 5 nearest neighbours:

Point	1	2	Triangle	3	4	5
Class	0	0	?	1	1	1
Distance	0.5	0.4	-	1.0	0.8	1.5
Inverse of dist.	$\frac{1}{0.5}$	$\frac{1}{0.4}$	-	$\frac{1}{1}$	$\frac{1}{0.8}$	$\frac{1}{1.5}$
WNN count	2	2.5	?	1	1.25	0.67

- KNN: $N_0 = 2$; $N_1 = 3 \therefore$ output class = 1
- WNN: max sum $N_0 = 2 + 2.5 = 4.5$; $N_1 = 1 + 1.25 + 0.67 = 2.92$, $\therefore$ output class = 0

KNN and WNN: Probabilities

- We're also interested in knowing the probability that a given input belongs in each possible class. More formally, it would be nice to know $P[C_i|\mathbf{x}]$, where C_i denotes the i-th class and $\mathbf{x}$ is the vector of input features.
- This expression is read:
"The probability of the correct class being C_i given that the observed features are x ".
- It is what's called a **conditional probability**.
- In both KNN and WNN, we can argue that

$$P[C_i|\mathbf{x}] = \frac{N_i}{\sum_k N_k}$$

where N_i is either the weighted or unweighted count of the nearest neighbours in class C_i .

KNN and WNN: Probabilities

- To continue with the last example, the WNN estimate of the probabilities of the two classes are:

$$P[C_0|\mathbf{x}] = \frac{4.5}{4.5+2.92} = 0.606 \text{ AND } P[C_1|\mathbf{x}] = \frac{2.92}{4.5+2.92} = 0.394$$

- The KNN estimates of the probabilities of the same events are:

$$P_{knn}[C_0|\mathbf{x}] = \frac{2}{5} = 0.4 \text{ AND } P_{knn}[C_1|\mathbf{x}] = \frac{3}{5} = 0.6$$

- If we use some other classifier, the estimated probabilities will be different again. Hard to argue that one result is more accurate than another.

Normalisation

- Another problem is that the dynamic range of one feature may be vastly different from that of another. E.g.
 1. Engine capacity (in cc) versus model year;
 2. Age (in years) versus height (in cm);
 3. Bit rate (in bps) versus transmitted power (in mW); etc.

So we should normalise to a standard scale, e.g. min-max normalisation:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

This maps the original values to a new range, usually between 0 and 1.

The function `preprocessing.sklearn.normalize` implements this normalisation algorithm.

KNN and WNN in Python

In **sklearn**, we can easily execute KNN and WNN using the **KneighborsClassifier** function.

Import required
libraries:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_score, recall_score, accuracy_score # For t
```

Define classifier
objects:

```
# Instantiate KNN and WNN classifiers
neigh1 = KNeighborsClassifier(n_neighbors = K, weights = "uniform")
neigh2 = KNeighborsClassifier(n_neighbors = K, weights = "distance")
```

Fit the model using
training data, and test it
using test data

```
# Pass training data into the two classifiers using the "fit" method
neigh1.fit(Xtrain,ytrain)
neigh2.fit(Xtrain,ytrain)

yhat1 = neigh1.predict(Xtest) # Use KNN to predict class labels in test data.
yhat2 = neigh2.predict(Xtest) # Use WNN
```

Scikit-Learn for Supervised Learning

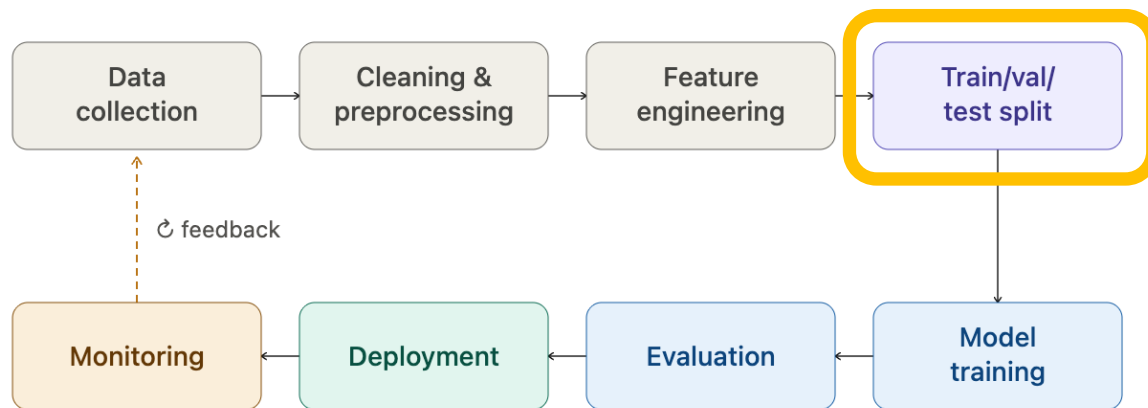
- In almost all supervised ML functions in `sklearn`, we take the following steps:
 1. Create an object for the desired ML algorithm using the appropriate class;
 2. Use the `fit` method to train the model;
 3. Use the `predict` method to estimate the class of the test data samples;
 4. Use the `predict_proba` method to estimate the probability of each class given the input features;
 5. Test the accuracy of the algorithm using one or more of the `metrics`.

KNN/WNN in the General ML Model

- Remember that we said an ML model is a function $f_{\theta}(x)$ that outputs the class label corresponding to the input variables x , where θ is the parameter vector.
- In KNN and WNN, there are no obvious parameters to optimise; instead the training data is used to create decision regions.
- We can consider the decision boundaries (i.e. borders between adjacent decision regions) to be the “parameters”.
- The parameter K (no. of nearest neighbours) is a hyperparameter that determines the exact form of the model.
- Hyperparameters are not learned or updated during the optimisation process. They are set before training begins. They are fixed values that determine the overall architecture and behaviour of the model.

More on Validation and Testing

Training, Validation and Testing Split



gray = data prep · purple = split · blue = train/evaluate · teal = deploy · amber = monitor

Validation

- In this step, labelled data that has not been used in training is used to determine how good the current model is.
 - Hyper-parameters of the model, and even the model itself, are adjusted and iterated in a training-validation cycle until the designer is satisfied.
- Often, validation is not done just once for each setting but instead performance is averaged over multiple validation steps, called **cross-validation**.
 - Most commonly, we use K-fold cross-validation.
 - See five-fold cross-validation illustration on next page.
 - Implemented using

```
from sklearn.model_selection import StratifiedKFold  
cv = StratifiedKFold(n_splits = 5)
```

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Iteration 1	val	train	train	train	train
Iteration 2	train	val	train	train	train
Iteration 3	train	train	val	train	train
Iteration 4	train	train	train	val	train
Iteration 5	train	train	train	train	val

final score = average of the 5 validation scores

Testing After Validation

- Once the right model is chosen after validation, the final step is to test it on the final set of data that has not been used, the test set.
- The performance of the trained and validated model on the test dataset is what should be reported.
 - This is the performance we expect after deployment, when data that has not been seen in training or validation is presented to the model.

What's Coming Up?

Next Week's Class

Week 4 Preview

- Focus is on neural networks, the most important class of AI/ML model in use today
 - Basic concepts
 - Different sorts of NNs
 - Why differentiability is key
 - Example applications

See you next week!



ENGG2112 week 4



Survey on this lecture